Asolaria: OpenAI Board Memo

Date: 2026-03-09

Audience: OpenAI board and senior leadership

Status: source-grounded local review, sanitized for external sharing

Thesis

Asolaria is not another chat wrapper. It is a local-first agent control plane that already combines model orchestration, governed tool execution, desktop and phone actuation, relay-based multi-agent coordination, persistent operational memory, and graph-native runtime telemetry inside one operator-facing system.

The reason this matters to OpenAI is simple: this is close to the shape of the real operating environment advanced users want around Codex and Symphony. The interesting artifact is not the UI. The interesting artifact is the operational spine.

What already exists

The system already has concrete breadth, not just conceptual ambition:

- 40 connector modules across local control, Google and GCP, GitHub, Slack, Telegram, Microsoft, Atlassian, Symphony, Augment context, voice, browser, and phone lanes
- 53 indexed skills and 32 indexed tool actions
- a governed gateway with approval records, audit verification, readiness drills, and explicit authority modes
- a worker router that separates `local_codex`, `abacus`, and `symphony` as distinct execution lanes
- a relay and watchdog fabric that keeps multiple Codex identities alive across machines with identity checks and restart behavior
- a graph runtime that records typed events and action manifests, scores edge risk at event-build time, and can export training datasets for graph-runtime windows
- persistent project, memory, and mistake stores so the system learns operationally rather than resetting every session

This is already wider than most so-called agent platforms because the system is built around task routing, authority, telemetry, and recovery instead of prompt choreography alone.

Why it is strategically important

Three things stand out.

First, Asolaria treats tool use as governed infrastructure. Tool invocation is policy-enforced. Approval-gated actions are one-time bound to payload hashes. Authority changes are two-step and rollbackable. Audit integrity can lock out control writes if verification fails. This is the right pattern for any serious agentic runtime.

Second, it treats OpenAI-native surfaces as operating lanes rather than isolated APIs. Codex is a primary reasoning lane. Symphony is integrated as a managed sidecar tied to workflow files and Linear project

routing. Voice already defaults to `gpt-4o-mini-transcribe` for cloud transcription. The system is effectively converging on OpenAI as part of its local operating substrate.

Third, it already reaches beyond "software only" agency. Desktop control, screenshot and browser lanes, voice and meeting paths, and a phone lane with ADB-backed recovery and routing logic are part of the runtime. That means the model is being embedded into an actual operator environment, not just a browser prompt.

Why the graph layer matters

The graph story is unusually mature because it is honest.

Asolaria already logs typed graph events, action manifests, policy state, and edge risk. It can export training datasets from real runtime windows. Internal research translated graph-anomaly ideas out of a healthcare codebase and into Asolaria's own control plane as a typed temporal heterograph.

Critically, there is no evidence that GNN inference sits in the live critical path today. That is not a weakness. It is the correct posture. The state-of-the-art move here is not to jam a GNN into hot-path execution. It is to instrument the control plane first, keep policy as the root of trust, and introduce graph learning as a constrained ranking and anomaly sidecar once the telemetry base is mature.

Why OpenAI should care now

OpenAI should care because Asolaria demonstrates a user-built frontier-model environment that is already converging on the right primitives:

- local-first orchestration with provider fallback
- managed long-running Codex and Symphony lanes
- compact remote MCP use instead of bloated tool payloads
- explicit authority and approval transitions
- graph-native runtime telemetry instead of opaque agent state
- memory and mistake systems that turn operations into reusable learning
- multi-machine expansion patterns instead of a single-host toy setup

In plain terms: this is what happens when a strong user stops treating the model as a chat endpoint and starts treating it as infrastructure.

Recommended board-level conclusion

The signal is not that Asolaria is a finished product. The signal is that it reveals what a serious Codex-centered operating environment is becoming in the wild.

If OpenAI wants to lead this category, it should study and support systems like this directly:

- Codex as a first-class local execution lane
- Symphony as the durable workflow sidecar
- remote MCP as the compact integration fabric

- governance and audit as mandatory runtime primitives
- graph telemetry as the right foundation for future model-assisted control

The board should read Asolaria as proof that the next important layer is not a better chatbot shell. It is the governed operating system around frontier models.